Esercizi sui thread in C

Guida pratica: istruzioni, schemi pronti e tracce svolte

Sistemi Operativi e Reti di Calcolatori

Ingegneria Informatica e Automatica, 3° anno

Per cosa serve	Svolgere gli esercizi di programmazione con pthread e semafori: dalla traccia alla soluzione.
Dentro trovi	Metodo in 7 passi, tabella "se la traccia dice… usa…", riferimento delle istruzioni, schemi da copiare, 6 tracce svolte, errori tipici e checklist.
Codice	Tutto compilato con gcc -Wall -Wextra -pthread e provato su Linux.
Navigazione	Indice cliccabile, tabella "cerco → vado a", segnalibri del PDF e link [sezione] nel testo.

Indice

1. Come si affronta una traccia 4

1.1 Il metodo in 7 passi 4

1.2 "Se la traccia dice... usa..." 4

1.3 Le 4 domande per ogni dato condiviso 5

2. Scheletro, argomenti e compilazione 6

2.1 Scheletro di partenza 6

2.2 Argomenti da riga di comando 7

2.3 Compilare ed eseguire 7

3. Le istruzioni: riferimento pratico 8

3.1 Thread 8

3.2 Mutex 8

3.3 Variabili di condizione 9

3.4 Semafori POSIX 9

3.5 Rwlock, barrier, once 9

3.6 Detach, attributi, cancellazione 10

3.7 Funzioni di contorno che servono nelle tracce 10

3.8 Codici d'errore più comuni 10

4. Schemi pronti da copiare 11

4.1 N thread con argomenti distinti 11

4.2 Restituire un risultato 11

4.3 Dati condivisi + mutex nella stessa struct 12

4.4 Attendere una condizione 12

4.5 Coda limitata (produttori e consumatori) 12

4.6 Semafori: ordine, risorse, mutua esclusione 13

4.7 Più lock insieme senza deadlock 13

4.8 Attesa con timeout 14

4.9 Uscire dalla sezione critica in un solo punto 14

5. Tracce svolte 15

5.1 E1 — Massimo e numero di pari di un array (N thread) 15

5.2 E2 — M produttori e K consumatori con coda limitata 17

5.3 E3 — Barriera costruita con mutex e variabile di condizione 19

5.4 E4 — Tre thread che stampano A B C in ordine (staffetta) 20

5.5 E5 — Trasferimenti tra conti senza deadlock 22

5.6 E6 — Server TCP: un thread per client e statistiche condivise 23

6. Errori tipici, checklist e test 26

6.1 I 15 errori che costano più punti 26

6.2 Checklist prima di consegnare 26

6.3 Come provare che funziona 27

6.4 Da saper spiegare se ti chiedono "perché?" 27

6.5 Appendice — File e Makefile 27

Cerco... vado a

Cerco...	Vai a
Da dove comincio con una traccia nuova	1.1
Quale strumento uso per questa richiesta	1.2
Scheletro del programma, leggere `argv`	2.1, 2.2
Sintassi di thread, mutex, condizioni, semafori	Cap. 3
Passare dati ai thread e prendere i risultati	4.1, 4.2
Attendere una condizione, timeout	4.4, 4.8
Coda produttori/consumatori da copiare	E2 (5.2)
Ordinare i thread (A B C), turni, risorse limitate	E4, 4.6
Barriera, fasi	E3, 3.5
Più lock insieme, deadlock	E5, 4.7
Server TCP con thread	E6
Il programma si blocca o dà risultati diversi	6.1, 6.3
Checklist prima di consegnare	6.2

1. Come si affronta una traccia

1.1 Il metodo in 7 passi

1. **Leggi la traccia due volte** e sottolinea: quanti thread, cosa fa ciascuno, cosa si stampa, come finisce il programma, quali argomenti arrivano da riga di comando.
2. **Elenca i dati condivisi** (variabili, array, code, contatori, file descriptor) e chiediti per ciascuno: *qualcuno lo modifica mentre altri lo usano?* Se sì, serve protezione.
3. **Elenca le attese**: un thread deve aspettare che succeda qualcosa ("buffer non vuoto", "è il mio turno", "tutti sono arrivati")? Quella è una variabile di condizione o un semaforo.
4. **Scegli gli strumenti** con la tabella 1.2 e scrivi come commento *quale mutex protegge quali dati*.
5. **Parti dallo scheletro** (2.1): struct condivisa, array di argomenti, ciclo `create`, ciclo `join`.
6. **Scrivi prima il thread senza sincronizzazione**, poi aggiungi lock e condizioni dove servono.
7. **Verifica**: compila con `-Wall -Wextra -pthread`, esegui molte volte, prova con ThreadSanitizer (6.3).

1.2 "Se la traccia dice… usa…"

La traccia dice…	Strumento	Vai a
"N thread", "dividi il lavoro", "ogni thread elabora una parte"	`pthread_create` + `pthread_join`, blocchi distinti, risultato nella struct del thread	4.1, E1
"calcola/restituisci un valore"	campo `result` nella struct (o `malloc` + `join`)	4.2
contatore / lista / saldo "condiviso"	mutex nella stessa struct dei dati	4.3
"aspetta che…", "finché non…", "quando arriva…"	variabile di condizione, predicato in un `while`	4.4
buffer, coda, produttori e consumatori	coda limitata: mutex + 2 condition variable	E2
"termina quando…", chiusura, nessun altro dato	flag `closed` + `broadcast`	E2
"tutti devono arrivare prima di proseguire", fasi	barriera (`pthread_barrier_*` o a mano)	E3, 3.5
"prima A poi B", "alterna", "in ordine"	semafori inizializzati a 0 (staffetta)	E4
"al massimo N contemporaneamente", N risorse uguali	semaforo inizializzato a N	4.6
trasferimento tra due oggetti, più lock insieme	lock sempre nello stesso ordine (per id)	E5, 4.7
molti lettori e un solo scrittore	`pthread_rwlock_t`	3.5
"attesa massima", "timeout"	`timedwait` / `timedlock` con orario assoluto	4.8
server, "un thread per ogni client"	`accept` in ciclo + thread detached + `malloc` del descrittore	E6
thread "in background", nessuno ne attende la fine	`pthread_detach`	3.6

1.3 Le 4 domande per ogni dato condiviso

Domanda	Se la risposta è sì
Più thread lo toccano e almeno uno lo scrive?	Serve un mutex (o un'operazione atomica)
Un thread deve aspettare che cambi?	Variabile di condizione (con il **predicato** = quel dato) o semaforo
Un thread tiene un lock mentre ne chiede un altro?	Ordine fisso dei lock, altrimenti rischio deadlock
Chi decide che il lavoro è finito?	Un flag protetto dal mutex + `broadcast`, oppure i `join` in sequenza

DA ESAME

Il risultato **non deve mai dipendere dall'ordine** in cui i thread vengono eseguiti. Se la traccia richiede un ordine (A, B, C...), va imposto con semafori o condizioni, mai con `sleep`.

2. Scheletro, argomenti e compilazione

2.1 Scheletro di partenza

Va bene per la maggior parte delle tracce: **dati condivisi e mutex nella stessa struct**, **un argomento per ogni thread** in un array, controllo degli errori, ciclo di `create` e poi ciclo di `join`.

00_template.c

```c
/* 00_template.c - scheletro di partenza per un esercizio con thread
 * Compilare: gcc -Wall -Wextra -pthread 00_template.c -o 00
 */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <pthread.h>

#define N_THREADS 4

/* ---- dati condivisi + il mutex che li protegge, insieme ---- */
typedef struct {
    long            total;
    pthread_mutex_t lock;
} shared_t;

/* ---- argomenti PER OGNI thread ---- */
typedef struct {
    int       id;
    shared_t *shared;
} targ_t;

void *worker(void *arg) {
    targ_t *a = (targ_t *)arg;

    /* lavoro locale al thread (nessun lock) */
    long local = (long)a->id * 10;

    /* aggiornamento dei dati condivisi: lock solo qui */
    pthread_mutex_lock(&a->shared->lock);
    a->shared->total += local;
    pthread_mutex_unlock(&a->shared->lock);

    return NULL;
}

int main(void) {
    shared_t shared = { .total = 0 };
    pthread_mutex_init(&shared.lock, NULL);

    pthread_t tid[N_THREADS];
    targ_t    args[N_THREADS];

    for (int i = 0; i < N_THREADS; i++) {
        args[i].id     = i;
        args[i].shared = &shared;
        int rc = pthread_create(&tid[i], NULL, worker, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            exit(EXIT_FAILURE);
        }
    }

    for (int i = 0; i < N_THREADS; i++)
        pthread_join(tid[i], NULL);

    printf("totale = %ld\n", shared.total);
    pthread_mutex_destroy(&shared.lock);
    return 0;
}
```

2.2 Argomenti da riga di comando

Molte tracce passano il numero di thread o la dimensione dei dati con `argv`. Controlla **sempre** `argc` e che il numero sia valido: `atoi` non segnala errori, `strtol` sì.

Lettura sicura di un intero positivo

```
long parse_positive(const char *str, const char *name) {
    char *end;
    long x = strtol(str, &end, 10);
    if (*str == '\0' || *end != '\0' || x <= 0) {
        fprintf(stderr, "%s deve essere un intero positivo (ricevuto \"%s\")\n", name, str);
        exit(EXIT_FAILURE);
    }
    return x;
}

/* in main:
   if (argc != 3) { fprintf(stderr, "uso: %s n_elementi n_thread\n", argv[0]); return EXIT_FAILURE; }
   long n  = parse_positive(argv[1], "n_elementi");
   long nt = parse_positive(argv[2], "n_thread");
   pthread_t *tid = malloc(nt * sizeof(pthread_t));    // array allocati a runtime */
```

CONSIGLIO

Con un numero di thread variabile, alloca gli array con `malloc` (o `calloc`) e **fai** `free` alla fine. Un array `pthread_t tid[nt]` di dimensione variabile (VLA) funziona ma con numeri grandi può far esplodere lo stack.

Se i thread possono essere più degli elementi, limita: `if (nt > n) nt = n;`.

Nello zip allegato c'è anche un `Makefile`: `make` compila tutti gli esempi con `-Wall -Wextra -g -pthread`, `make clean` li cancella.

2.3 Compilare ed eseguire

Comandi utili

```
gcc -Wall -Wextra -g -pthread prog.c -o prog      # compilazione consigliata
./prog 1000000 4                                  # esecuzione
for i in $(seq 100); do ./prog; done | sort | uniq -c    # lo stesso output ogni volta?
timeout 5 ./prog; echo $?                         # 124 = bloccato (deadlock o attesa infinita)

gcc -g -O1 -fsanitize=thread -pthread prog.c -o prog_tsan && ./prog_tsan    # rileva data race
valgrind --tool=helgrind ./prog                   # race e ordine dei lock incoerente
```

ATTENZIONE

Se compili con `-std=c99` o `-std=c11` alcune funzioni (barrier, `usleep`, `rand_r`) possono non essere dichiarate: aggiungi `#define _GNU_SOURCE` (o `_POSIX_C_SOURCE 200809L`) **prima di tutti gli** `#include`. Senza `-pthread` ottieni `undefined reference to pthread_create`.

3. Le istruzioni: riferimento pratico

Per ogni gruppo: la riga da scrivere, cosa fa e **l'attenzione** che evita l'errore classico. Le funzioni `pthread_*` ritornano **0 se vanno bene, altrimenti il codice d'errore** (non -1, `errno` non impostato). Le `sem_*` ritornano **-1 e impostano** `errno`.

3.1 Thread

Istruzione	Cosa fa	Attenzione
`pthread_create(&tid, NULL, f, arg)`	Crea un thread che esegue `f(arg)`	`f` ha firma `void *f(void *)`; si passa il nome, non `f()`. `arg` deve restare valido
`pthread_join(tid, &ret)`	Attende la fine del thread e legge il valore ritornato (`ret` può essere `NULL`)	Una sola volta per thread; mai su un thread detached
`pthread_exit(val)` / `return val;`	Termina il thread chiamante	`val` non può essere `&variabile_locale`
`pthread_self()`	ID del thread corrente	Confronto con `pthread_equal(a, b)`, non `==`
`pthread_detach(tid)`	Il thread libera le risorse da solo	Poi non si può più fare `join`
`exit(n)` o `return` da `main`	Termina **tutto il processo**	I thread ancora attivi vengono uccisi

3.2 Mutex

Istruzione	Cosa fa	Attenzione
`pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;`	Mutex globale/static	Per i mutex dentro una struct o sull'heap usa `pthread_mutex_init(&m, NULL)`
`pthread_mutex_lock(&m)`	Entra nella sezione critica (blocca se occupato)	Tutti gli accessi al dato, anche in lettura
`pthread_mutex_unlock(&m)`	Esce dalla sezione critica	Su **ogni** percorso (`return`, `break`, errori)
`pthread_mutex_trylock(&m)`	Non blocca: 0 se preso, `EBUSY` se occupato	Utile per evitare deadlock
`pthread_mutex_destroy(&m)`	Libera il mutex	Mai su un mutex bloccato

ATTENZIONE

Sezione critica **breve**: niente `sleep`, `printf` lunghi, `recv` / `send` o calcoli pesanti con il lock preso.

3.3 Variabili di condizione

Istruzione	Cosa fa	Attenzione
`pthread_cond_wait(&cv, &m)`	Rilascia `m` e dorme (atomico); al risveglio riprende `m`	Solo con `m` preso; **sempre in un** `while`
`pthread_cond_signal(&cv)`	Sveglia un thread in attesa	Prima cambia lo stato, poi segnala
`pthread_cond_broadcast(&cv)`	Sveglia tutti	Per chiusure o condizioni diverse
`pthread_cond_timedwait(&cv, &m, &ts)`	Come `wait` con scadenza	`ts` è un orario **assoluto**; `ETIMEDOUT` se scade

Schema da sapere a memoria

```
/* ATTENDE */                                   /* SEGNALA */
pthread_mutex_lock(&m);                         pthread_mutex_lock(&m);
while (!condizione)                             condizione = 1;          /* 1) cambia lo stato */
    pthread_cond_wait(&cv, &m);                 pthread_cond_signal(&cv);/* 2) avvisa */
/* qui condizione vera e m preso */             pthread_mutex_unlock(&m);
pthread_mutex_unlock(&m);
```

3.4 Semafori POSIX

Istruzione	Cosa fa	Attenzione
`#include <semaphore.h>` — `sem_t s;`	Tipo semaforo	Si compila con `-pthread`
`sem_init(&s, 0, valore)`	Inizializza (0 = tra thread dello stesso processo)	`valore` iniziale: 1 mutex, 0 evento, N risorse
`sem_wait(&s)`	P: se il valore è 0 attende, altrimenti decrementa	Può dare `EINTR` se arriva un segnale
`sem_post(&s)`	V: incrementa e sveglia un thread	Lo può fare qualunque thread
`sem_trywait(&s)` / `sem_timedwait(&s, &ts)`	Non bloccante / con scadenza assoluta	`errno` = `EAGAIN` / `ETIMEDOUT`
`sem_destroy(&s)`	Distrugge il semaforo	Nessun thread deve essere in attesa

ATTENZIONE

Con più semafori l'**ordine delle** `sem_wait` **conta**: prima il semaforo contatore (`empty` / `full`), poi il mutex. Invertirli porta a deadlock.

3.5 Rwlock, barrier, once

Istruzione	Cosa fa
`pthread_rwlock_rdlock(&rw)` / `pthread_rwlock_wrlock(&rw)` / `pthread_rwlock_unlock(&rw)`	Molti lettori insieme, oppure uno scrittore in esclusiva. Init: `PTHREAD_RWLOCK_INITIALIZER`
`pthread_barrier_init(&b, NULL, N)` / `pthread_barrier_wait(&b)` / `pthread_barrier_destroy(&b)`	Nessuno prosegue finché non sono arrivati N thread. Riutilizzabile. Serve `_GNU_SOURCE` se compili con `-std=`
`pthread_once(&once, fn)` con `pthread_once_t once = PTHREAD_ONCE_INIT;`	`fn` viene eseguita una sola volta
`__thread int x;` oppure `_Thread_local int x;`	Variabile con una copia per ogni thread

3.6 Detach, attributi, cancellazione

Istruzione	Cosa fa
`pthread_detach(tid)`	Thread "spara e dimentica": libera le risorse da solo. Ideale per un thread per client
`pthread_attr_init(&a)` + `pthread_attr_setdetachstate(&a, PTHREAD_CREATE_DETACHED)` + `pthread_attr_destroy(&a)`	Crea il thread già detached (si passa `&a` come secondo parametro di `create`)
`pthread_attr_setstacksize(&a, byte)`	Cambia la dimensione dello stack (default Linux: 8 MB)
`pthread_cancel(tid)`	Chiede la terminazione; agisce nei *cancellation point* (`sleep`, `read`, `cond_wait`, `sem_wait` ...)
`pthread_cleanup_push(fn, arg)` / `pthread_cleanup_pop(1)`	Esegue `fn` se il thread viene cancellato (es. rilascia un mutex). Stesso blocco!

CONSIGLIO

Per fermare un thread è più sicuro uno schema cooperativo: una variabile `stop` (protetta da mutex) che il thread controlla a ogni giro, invece di `pthread_cancel`.

3.7 Funzioni di contorno che servono nelle tracce

Istruzione	Uso
`malloc(n * sizeof(T))`, `calloc(n, sizeof(T))`, `free(p)`	Array dinamici; controlla `NULL`, libera alla fine
`strtol(s, &end, 10)`	Conversione sicura stringa→intero (vedi 2.2)
`rand_r(&seed)`	Numeri casuali **thread-safe**: un `seed` privato per ogni thread (`rand()` condivide lo stato)
`usleep(us)`, `nanosleep(&ts, NULL)`, `sleep(s)`	Pausa: solo per simulare lavoro, **mai** per sincronizzare
`clock_gettime(CLOCK_REALTIME, &ts)`	Orario attuale; per i timeout aggiungi i secondi a `ts.tv_sec`
`snprintf(buf, sizeof(buf), ...)`	Formattazione sicura nei buffer
`strerror(rc)` / `perror("...")`	`strerror` per i codici di `pthread_*`; `perror` per `errno` (system call, `sem_*`)
`atomic_fetch_add(&x, 1)` con `<stdatomic.h>`	Incremento atomico di `atomic_long` senza mutex

3.8 Codici d'errore più comuni

Codice	Quando
`EAGAIN`	`pthread_create`: troppi thread/risorse; `sem_trywait`: semaforo a 0
`EINVAL`	Argomento non valido (oggetto non inizializzato, thread non joinable...)
`EBUSY`	`trylock` su un mutex occupato; `destroy` su un oggetto in uso
`ETIMEDOUT`	Scadenza di `timedwait` / `timedlock` / `sem_timedwait`
`EDEADLK`, `EPERM`, `ESRCH`	Join su se stessi o lock ricorsivo su mutex *errorcheck*; `unlock` di un mutex non posseduto; thread inesistente
`EINTR`	`sem_wait` / `recv` / `accept` interrotte da un segnale: riprovare

4. Schemi pronti da copiare

Frammenti pensati per essere incollati e adattati. Tutti compilano senza warning con `-Wall -Wextra -pthread`.

4.1 N thread con argomenti distinti

```c
#define N_THREADS 4

typedef struct {
    int id;                     /* altri campi: puntatori ai dati condivisi, intervallo, risultato... */
} arg_t;

void *worker4(void *arg) {
    arg_t *a = (arg_t *)arg;
    (void)a;                    /* ...lavoro del thread... */
    return NULL;
}

void run_n_threads(void) {
    pthread_t tid[N_THREADS];
    arg_t     args[N_THREADS];  /* UN elemento per thread: mai &i della variabile del ciclo */

    for (int i = 0; i < N_THREADS; i++) {
        args[i].id = i;
        int rc = pthread_create(&tid[i], NULL, worker4, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            exit(EXIT_FAILURE);
        }
    }
    for (int i = 0; i < N_THREADS; i++)
        pthread_join(tid[i], NULL);
}
```

4.2 Restituire un risultato

Tre modi, dal più semplice al più raro. Il primo va bene quasi sempre.

```c
typedef struct { int id; long result; } task_t;

void *calc(void *arg) {                         /* 1) scrive nella struct passata */
    task_t *task = (task_t *)arg;
    task->result = (long)task->id * 2;
    return NULL;
}

void *calc_ret(void *arg) {                     /* 2) ritorna memoria allocata */
    long *r = malloc(sizeof(long));
    *r = (long)(intptr_t)arg * 2;
    return r;
}

void *calc_int(void *arg) {                     /* 3) ritorna un intero "mascherato" da puntatore */
    return (void *)((intptr_t)arg * 2);
}

void use_results(void) {
    pthread_t th;
    void *ret;

    task_t task = { .id = 3 };
    pthread_create(&th, NULL, calc, &task);
    pthread_join(th, NULL);
    printf("%ld\n", task.result);               /* letto DOPO il join */

    pthread_create(&th, NULL, calc_ret, (void *)(intptr_t)5);
    pthread_join(th, &ret);
    printf("%ld\n", *(long *)ret);
    free(ret);                                  /* chi riceve libera */
```

```
    pthread_create(&th, NULL, calc_int, (void *)(intptr_t)7);
    pthread_join(th, &ret);
    printf("%ld\n", (long)(intptr_t)ret);
}
```

4.3 Dati condivisi + mutex nella stessa struct

Ogni thread riceve un puntatore alla stessa struct (campo `shared` nel suo argomento)

```
typedef struct {
    long            total;       /* dato condiviso... */
    int             done;
    pthread_mutex_t lock;        /* ...e il mutex che lo protegge, insieme */
    pthread_cond_t  cv;
} shared_data_t;

void shared_init(shared_data_t *sd) {
    sd->total = 0;
    sd->done  = 0;
    pthread_mutex_init(&sd->lock, NULL);
    pthread_cond_init(&sd->cv, NULL);
}

void shared_add(shared_data_t *sd, long x) {
    pthread_mutex_lock(&sd->lock);
    sd->total += x;
    pthread_mutex_unlock(&sd->lock);
}
```

4.4 Attendere una condizione

`m`, `cv` e `ready` sono globali: dichiarati e inizializzati con i macro `*_INITIALIZER`

```
void wait_for_ready(void) {
    pthread_mutex_lock(&m);
    while (!ready)                      /* while, mai if */
        pthread_cond_wait(&cv, &m);
    /* qui ready e' vero e il mutex e' preso */
    pthread_mutex_unlock(&m);
}

void set_ready(void) {
    pthread_mutex_lock(&m);
    ready = 1;                          /* 1) prima lo stato... */
    pthread_cond_broadcast(&cv);        /* 2) ...poi l'avviso */
    pthread_mutex_unlock(&m);
}
```

4.5 Coda limitata (produttori e consumatori)

È lo schema più frequente. L'implementazione completa e provata (`bq_init` , `bq_push` , `bq_pop` , `bq_close`) è nella traccia E2 e si copia così com'è. Come si usa:

Schema di terminazione: la chiusura dopo il join dei produttori sveglia i consumatori

```
bq_init(&queue);
/* crea consumatori e produttori */
/* produttore:   while (ho dati) bq_push(&queue, x);                       */
/* consumatore:  while (bq_pop(&queue, &x) == 0) { usa x; }                */
/* main:  join dei PRODUTTORI  ->  bq_close(&queue)  ->  join dei CONSUMATORI */
```

> **DA ESAME**
>
> Con **più consumatori** non puoi far terminare ogni consumatore dopo "N elementi": usa la chiusura della coda (o un elemento sentinella per consumatore).

4.6 Semafori: ordine, risorse, mutua esclusione

Obiettivo	Valore iniziale	Schema
Mutua esclusione	1	`sem_wait` ... sezione critica ... `sem_post`
Il thread B parte dopo che A ha finito	0	A: `sem_post(&s)` — B: `sem_wait(&s)`
Al massimo N thread contemporaneamente	N	`sem_wait` per entrare, `sem_post` per uscire
Staffetta A→B→C→A...	1 al primo, 0 agli altri	ognuno: `sem_wait(&mio)` ... `sem_post(&successivo)` (vedi E4)

```c
sem_t a_done;                           /* sem_init(&a_done, 0, 0) in main */
sem_t slots;                            /* sem_init(&slots, 0, 3): max 3 insieme */

void *first_task(void *arg) {
    (void)arg;
    puts("A");
    sem_post(&a_done);                  /* avviso: A e' finito */
    return NULL;
}

void *second_task(void *arg) {
    (void)arg;
    sem_wait(&a_done);                  /* B aspetta A */
    puts("B");
    return NULL;
}

void use_limited_resource(void) {
    sem_wait(&slots);                   /* al massimo 3 thread qui dentro */
    /* ...uso della risorsa... */
    sem_post(&slots);
}
```

4.7 Più lock insieme senza deadlock

Regola: **tutti i thread prendono i lock nello stesso ordine**. Il modo più semplice è per identificativo (come in E5); se non hai un id, ordina per indirizzo:

```c
void lock_two(pthread_mutex_t *a, pthread_mutex_t *b) {
    if (a == b) {                       /* stesso mutex: UNA sola lock */
        pthread_mutex_lock(a);
        return;
    }
    if ((uintptr_t)a > (uintptr_t)b) {  /* ordine globale: indirizzo piu' basso prima */
        pthread_mutex_t *tmp = a; a = b; b = tmp;
    }
    pthread_mutex_lock(a);
    pthread_mutex_lock(b);
}

void unlock_two(pthread_mutex_t *a, pthread_mutex_t *b) {
    pthread_mutex_unlock(a);
    if (a != b) pthread_mutex_unlock(b);
}
```

4.8 Attesa con timeout

```
int wait_ready_2s(void) {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += 2;                              /* orario ASSOLUTO: ora + 2 secondi */

    int rc = 0;
    pthread_mutex_lock(&m);
    while (!ready && rc != ETIMEDOUT)
        rc = pthread_cond_timedwait(&cv, &m, &ts);
    int ok = ready;
    pthread_mutex_unlock(&m);
    return ok;                                   /* 1 = condizione vera, 0 = timeout */
}
```

4.9 Uscire dalla sezione critica in un solo punto

Utile quando ci sono molti `return`/`break` dentro la sezione critica

```
int take_item(int *queue_size) {
    int rc = -1;
    pthread_mutex_lock(&m);
    if (*queue_size == 0)
        goto out;                                /* l'unlock avviene comunque */
    (*queue_size)--;
    rc = 0;
out:
    pthread_mutex_unlock(&m);
    return rc;
}
```

5. Tracce svolte

Sei esercizi tipici, ognuno con **traccia, ragionamento, codice completo e verifica**. Tutti compilati con `gcc -Wall -Wextra -pthread` e provati; i sorgenti sono nello zip allegato. Per ogni traccia c'è anche "Se la traccia cambia...": le varianti più probabili.

5.1 E1 — Massimo e numero di pari di un array (N thread)

> **NOTA**
>
> **Traccia.** Il programma riceve da riga di comando il numero di elementi `n` e il numero di thread `nt`. Riempie un array di `n` interi casuali e crea `nt` thread: ognuno analizza una porzione e calcola massimo e quantità di numeri pari. Il main combina i risultati parziali e li stampa.

Come ragionare

- L'array è **condiviso ma in sola lettura**: nessun mutex.
- Ogni thread scrive i risultati nella **sua** struct (`job_t`): nessuna scrittura condivisa, nessun lock.
- Il main unisce **dopo il** `join`, che garantisce anche la visibilità dei valori scritti.
- Divisione del lavoro: `base = n / nt` elementi a testa, e i primi `n % nt` thread ne prendono uno in più (nessun elemento perso).
- Controlli: argomenti validi e `nt <= n`.

E1_max_parallelo.c

```c
/* E1_max_parallelo.c - massimo e numero di pari di un array, diviso tra N thread
 * Uso:       ./E1 n_elementi n_thread
 * Compilare: gcc -Wall -Wextra -pthread E1_max_parallelo.c -o E1
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <pthread.h>

typedef struct {
    const int *v;        /* array condiviso, in SOLA LETTURA: niente lock */
    long start, end;     /* blocco [start, end) */
    int  max;            /* risultati parziali: ogni thread scrive solo nella SUA struct */
    long even;
} job_t;

static void *scan(void *arg) {
    job_t *j = (job_t *)arg;
    j->max  = INT_MIN;
    j->even = 0;
    for (long i = j->start; i < j->end; i++) {
        if (j->v[i] > j->max) j->max = j->v[i];
        if (j->v[i] % 2 == 0) j->even++;
    }
    return NULL;
}

static long parse_positive(const char *s, const char *name) {
    char *end;
    long x = strtol(s, &end, 10);
    if (*s == '\0' || *end != '\0' || x <= 0) {
        fprintf(stderr, "%s deve essere un intero positivo (ricevuto \"%s\")\n", name, s);
        exit(EXIT_FAILURE);
    }
    return x;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "uso: %s n_elementi n_thread\n", argv[0]);
```

```
        return EXIT_FAILURE;
    }
    long n  = parse_positive(argv[1], "n_elementi");
    long nt = parse_positive(argv[2], "n_thread");
    if (nt > n) nt = n;                              /* mai piu' thread che elementi */

    int       *v    = malloc(n  * sizeof(int));
    pthread_t *tid  = malloc(nt * sizeof(pthread_t));
    job_t     *jobs = malloc(nt * sizeof(job_t));
    if (!v || !tid || !jobs) { perror("malloc"); return EXIT_FAILURE; }

    srand(42);
    for (long i = 0; i < n; i++) v[i] = rand() % 100000;

    /* divisione equa: i primi (n % nt) thread prendono un elemento in piu' */
    long base = n / nt, rem = n % nt, pos = 0;
    for (long i = 0; i < nt; i++) {
        jobs[i].v     = v;
        jobs[i].start = pos;
        pos          += base + (i < rem ? 1 : 0);
        jobs[i].end   = pos;
        int rc = pthread_create(&tid[i], NULL, scan, &jobs[i]);
        if (rc != 0) { fprintf(stderr, "pthread_create: %s\n", strerror(rc)); return EXIT_FAILURE; }
    }

    int  gmax  = INT_MIN;
    long geven = 0;
    for (long i = 0; i < nt; i++) {
        pthread_join(tid[i], NULL);                  /* dopo il join i campi sono visibili */
        if (jobs[i].max > gmax) gmax = jobs[i].max;
        geven += jobs[i].even;
    }

    /* verifica sequenziale */
    int  smax = INT_MIN;
    long seven = 0;
    for (long i = 0; i < n; i++) {
        if (v[i] > smax) smax = v[i];
        if (v[i] % 2 == 0) seven++;
    }

    printf("thread=%ld  max=%d  pari=%ld\n", nt, gmax, geven);
    printf("verifica sequenziale: %s\n", (gmax == smax && geven == seven) ? "OK" : "DIVERSA!");

    free(v); free(tid); free(jobs);
    return 0;
}
```

Output verificato

```
$ ./E1 1000000 4
thread=4  max=99999  pari=501086
verifica sequenziale: OK
$ ./E1 abc 4
n_elementi deve essere un intero positivo (ricevuto "abc")
```

CONSIGLIO

Se la traccia cambia… *Somma/media*: stesso schema, campo `long sum` . *Matrice*: dividi per righe. *Un solo totale condiviso*: accumula in locale e aggiorna il totale **una volta** a fine lavoro con un mutex.

5.2 E2 — M produttori e K consumatori con coda limitata

> **NOTA**
>
> **Traccia.** Più produttori inseriscono numeri in un buffer circolare di capienza fissa; più consumatori li prelevano e li sommano. Se il buffer è pieno i produttori attendono, se è vuoto i consumatori attendono. Alla fine il programma stampa quanti elementi sono stati consumati e la loro somma.

Come ragionare

- Dati condivisi: buffer, indici `head` / `tail`, `count`, flag `closed` → **un mutex** nella struct.
- Due attese diverse → **due condition variable**: `not_full` (per i produttori) e `not_empty` (per i consumatori).
- Le attese sono in un `while` con il **predicato** (`count == QCAP`, `count == 0`) e si controlla anche `closed`.
- Dopo aver cambiato lo stato si fa `signal` dell'altra condizione.
- **Terminazione**: il main fa `join` dei produttori, poi `bq_close` (flag + `broadcast`), poi `join` dei consumatori. `bq_pop` ritorna -1 solo se la coda è chiusa **e vuota**, così non si perde nessun elemento.
- Ogni consumatore accumula in una struct **sua**: nessun lock per le somme.

E2_coda_limitata.c

```c
/* E2_coda_limitata.c - M produttori e K consumatori con coda limitata e chiusura pulita
 * Coda circolare protetta da mutex + due variabili di condizione (not_full, not_empty).
 * La chiusura (flag "closed" + broadcast) permette ai consumatori di terminare senza sapere
 * in anticipo quanti elementi arriveranno.
 * Compilare: gcc -Wall -Wextra -pthread E2_coda_limitata.c -o E2
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define QCAP        8
#define N_PRODUCERS 3
#define N_CONSUMERS 2
#define ITEMS_EACH  1000            /* ogni produttore inserisce 1..ITEMS_EACH */

typedef struct {
    int             buf[QCAP];
    int             head, tail, count;
    int             closed;                 /* 1 = nessun nuovo elemento arrivera' */
    pthread_mutex_t mtx;
    pthread_cond_t  not_full;
    pthread_cond_t  not_empty;
} bqueue_t;

void bq_init(bqueue_t *q) {
    q->head = q->tail = q->count = q->closed = 0;
    pthread_mutex_init(&q->mtx, NULL);
    pthread_cond_init(&q->not_full, NULL);
    pthread_cond_init(&q->not_empty, NULL);
}

/* 0 = inserito, -1 = coda chiusa */
int bq_push(bqueue_t *q, int x) {
    pthread_mutex_lock(&q->mtx);
    while (q->count == QCAP && !q->closed)
        pthread_cond_wait(&q->not_full, &q->mtx);
    if (q->closed) {
        pthread_mutex_unlock(&q->mtx);
        return -1;
    }
    q->buf[q->tail] = x;
    q->tail = (q->tail + 1) % QCAP;
    q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->mtx);
    return 0;
```

```c
}

/* 0 = elemento letto in *x, -1 = coda chiusa E vuota (il consumatore puo' terminare) */
int bq_pop(bqueue_t *q, int *x) {
    pthread_mutex_lock(&q->mtx);
    while (q->count == 0 && !q->closed)
        pthread_cond_wait(&q->not_empty, &q->mtx);
    if (q->count == 0) {                    /* qui closed e' per forza 1 */
        pthread_mutex_unlock(&q->mtx);
        return -1;
    }
    *x = q->buf[q->head];
    q->head = (q->head + 1) % QCAP;
    q->count--;
    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->mtx);
    return 0;
}

void bq_close(bqueue_t *q) {
    pthread_mutex_lock(&q->mtx);
    q->closed = 1;
    pthread_cond_broadcast(&q->not_empty);   /* broadcast: svegliare TUTTI i consumatori */
    pthread_cond_broadcast(&q->not_full);
    pthread_mutex_unlock(&q->mtx);
}

/* ---------- produttori e consumatori ---------- */
bqueue_t queue;

typedef struct { long sum; long n; } consumer_t;   /* un risultato PRIVATO per consumatore */

void *producer(void *arg) {
    (void)arg;
    for (int i = 1; i <= ITEMS_EACH; i++)
        if (bq_push(&queue, i) != 0) break;
    return NULL;
}

void *consumer(void *arg) {
    consumer_t *c = (consumer_t *)arg;
    int x;
    while (bq_pop(&queue, &x) == 0) {
        c->sum += x;
        c->n++;
    }
    return NULL;
}

int main(void) {
    pthread_t  prod[N_PRODUCERS], cons[N_CONSUMERS];
    consumer_t res[N_CONSUMERS] = {{0, 0}};

    bq_init(&queue);
    for (int i = 0; i < N_CONSUMERS; i++) pthread_create(&cons[i], NULL, consumer, &res[i]);
    for (int i = 0; i < N_PRODUCERS; i++) pthread_create(&prod[i], NULL, producer, NULL);

    for (int i = 0; i < N_PRODUCERS; i++) pthread_join(prod[i], NULL);  /* 1) finiscono i produttori */
    bq_close(&queue);                                                   /* 2) si chiude la coda      */
    for (int i = 0; i < N_CONSUMERS; i++) pthread_join(cons[i], NULL);  /* 3) i consumatori escono   */

    long sum = 0, n = 0;
    for (int i = 0; i < N_CONSUMERS; i++) { sum += res[i].sum; n += res[i].n; }

    long exp_n   = (long)N_PRODUCERS * ITEMS_EACH;
    long exp_sum = (long)N_PRODUCERS * ITEMS_EACH * (ITEMS_EACH + 1) / 2;
    printf("elementi consumati = %ld (attesi %ld)\n", n, exp_n);
    printf("somma              = %ld (attesa %ld)\n", sum, exp_sum);
    printf("risultato: %s\n", (n == exp_n && sum == exp_sum) ? "OK" : "ERRORE");
    return 0;
}
```

Output verificato

```
elementi consumati = 3000 (attesi 3000)
somma              = 1501500 (attesa 1501500)
risultato: OK
```

CONSIGLIO

Se la traccia cambia… *Un solo consumatore*: nessun cambio. *Buffer di stringhe/struct*: cambia il tipo di `buf`. *Con semafori invece delle condition variable*: `empty_slots` = QCAP, `full_slots` = 0, `mutex` = 1 (prima il contatore, poi il mutex). *Sentinella*: il main inserisce K valori speciali (uno per consumatore) invece di chiudere.

5.3 E3 — Barriera costruita con mutex e variabile di condizione

NOTA

Traccia. Implementa una barriera riutilizzabile (`mb_init` , `mb_wait`) usando solo mutex e variabili di condizione. Verificala con N thread che eseguono P fasi: nessun thread deve iniziare la fase *p+1* prima che tutti abbiano finito la fase *p*.

Come ragionare

- Stato: quanti thread sono arrivati (`waiting`), quanti ne servono (`parties`).
- L'**ultimo** che arriva azzera `waiting` e fa `broadcast` ; gli altri dormono.
- Il trucco della **generazione** (`generation`): ogni thread ricorda il giro a cui appartiene e dorme finché `generation` non cambia. Senza, un thread veloce che rientra nella barriera potrebbe confondersi con il giro precedente (e i risvegli spuri romperebbero la barriera).
- L'attesa è un `while (my_gen == b->generation)` : il predicato non è `waiting` , è il numero di giro.

E3_barriera_cond.c

```c
/* E3_barriera_cond.c - barriera riutilizzabile costruita con mutex + variabile di condizione
 * Compilare: gcc -Wall -Wextra -pthread E3_barriera_cond.c -o E3
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    pthread_mutex_t mtx;
    pthread_cond_t  cv;
    int      parties;       /* quanti thread devono arrivare */
    int      waiting;       /* quanti sono arrivati in questo giro */
    unsigned generation;    /* numero del giro: distingue un giro dal successivo */
} mybarrier_t;

void mb_init(mybarrier_t *b, int parties) {
    pthread_mutex_init(&b->mtx, NULL);
    pthread_cond_init(&b->cv, NULL);
    b->parties    = parties;
    b->waiting    = 0;
    b->generation = 0;
}

void mb_wait(mybarrier_t *b) {
    pthread_mutex_lock(&b->mtx);
    unsigned my_gen = b->generation;                /* il giro a cui appartengo */
    if (++b->waiting == b->parties) {               /* sono l'ultimo: apro la barriera */
        b->waiting = 0;
        b->generation++;                            /* nuovo giro: riutilizzabile */
        pthread_cond_broadcast(&b->cv);
    } else {
        while (my_gen == b->generation)             /* while: protegge dai risvegli spuri */
            pthread_cond_wait(&b->cv, &b->mtx);
    }
    pthread_mutex_unlock(&b->mtx);
}
```

```c
/* ---------- prova: N thread, P fasi ---------- */
#define N_THREADS 4
#define PHASES    3

mybarrier_t     bar;
pthread_mutex_t check_mtx = PTHREAD_MUTEX_INITIALIZER;
int             arrived[PHASES];       /* quanti thread hanno finito la fase p */
int             violations = 0;

void *phased(void *arg) {
    long id = (long)arg;
    for (int p = 0; p < PHASES; p++) {
        /* lavoro della fase p */
        pthread_mutex_lock(&check_mtx);
        arrived[p]++;
        pthread_mutex_unlock(&check_mtx);

        mb_wait(&bar);

        /* dopo la barriera TUTTI devono aver finito la fase p */
        pthread_mutex_lock(&check_mtx);
        if (arrived[p] != N_THREADS) violations++;
        pthread_mutex_unlock(&check_mtx);
    }
    (void)id;
    return NULL;
}

int main(void) {
    pthread_t tid[N_THREADS];
    mb_init(&bar, N_THREADS);

    for (long i = 0; i < N_THREADS; i++)
        pthread_create(&tid[i], NULL, phased, (void *)i);
    for (int i = 0; i < N_THREADS; i++)
        pthread_join(tid[i], NULL);

    printf("violazioni della barriera: %d (atteso 0)\n", violations);
    return violations != 0;
}
```

Output verificato (20 esecuzioni consecutive, sempre 0)

```
violazioni della barriera: 0 (atteso 0)
```

CONSIGLIO

Se la traccia cambia… *Con la barriera di libreria*: `pthread_barrier_init(&b, NULL, N)` e `pthread_barrier_wait(&b)`. *Un solo thread esegue un passo di unione tra le fasi*: `pthread_barrier_wait` ritorna `PTHREAD_BARRIER_SERIAL_THREAD` a uno solo; nella versione a mano, fallo fare a chi vede `waiting == parties`.

5.4 E4 — Tre thread che stampano A B C in ordine (staffetta)

NOTA

Traccia. Tre thread stampano rispettivamente `A`, `B`, `C`. L'output deve essere sempre `A B C` ripetuto per 4 righe, **indipendentemente dall'ordine di creazione** dei thread.

Come ragionare

- È un problema di **ordinamento**: serve un "turno" per ogni thread.
- Un semaforo per thread, tutti a **0** tranne il primo (**1**): parte solo il thread 0.
- Ognuno fa `sem_wait` sul **proprio** semaforo, stampa, poi `sem_post` sul semaforo del **successivo** (in cerchio).
- Nessun mutex: ad ogni istante è attivo un solo thread, quindi `printf` non è in concorrenza.
- L'ordine di creazione è irrilevante (qui sono creati al contrario di proposito).

E4_anello_semafori.c

```c
/* E4_anello_semafori.c - N thread stampano A B C ... sempre in ordine (staffetta con semafori)
 * Ogni thread ha il SUO semaforo (il "turno"), inizializzato a 0 tranne il primo.
 * Compilare: gcc -Wall -Wextra -pthread E4_anello_semafori.c -o E4
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N      3
#define ROUNDS 4

sem_t turn[N];

void *printer(void *arg) {
    long id = (long)arg;
    for (int r = 0; r < ROUNDS; r++) {
        sem_wait(&turn[id]);                          /* aspetto il MIO turno */
        printf("%c%c", 'A' + (int)id, id == N - 1 ? '\n' : ' ');
        sem_post(&turn[(id + 1) % N]);                /* passo il testimone al successivo */
    }
    return NULL;
}

int main(void) {
    pthread_t tid[N];

    for (int i = 0; i < N; i++)
        sem_init(&turn[i], 0, i == 0 ? 1 : 0);       /* parte solo il thread 0 */

    for (long i = N - 1; i >= 0; i--)                /* creati "al contrario" di proposito: */
        pthread_create(&tid[i], NULL, printer, (void *)i);   /* l'ordine di creazione non conta */

    for (int i = 0; i < N; i++)
        pthread_join(tid[i], NULL);

    for (int i = 0; i < N; i++)
        sem_destroy(&turn[i]);
    return 0;
}
```

Output verificato (identico in 30 esecuzioni su 30)

```
A B C
A B C
A B C
A B C
```

CONSIGLIO

Se la traccia cambia… *Ping-pong tra due thread*: N = 2. *Un thread stampa due volte per giro*: fai due `sem_post` o usa un semaforo in più. *Con condition variable*: una variabile `turn` condivisa e `while (turn != id) wait` , poi `turn = (id + 1) % N` e `broadcast` .

5.5 E5 — Trasferimenti tra conti senza deadlock

> **NOTA**
>
> **Traccia.** Un array di conti correnti, ognuno con il suo saldo. Più thread eseguono molti trasferimenti casuali tra due conti. Il trasferimento avviene solo se il saldo è sufficiente. Alla fine il denaro totale deve essere invariato e il programma non deve bloccarsi.

Come ragionare

- Un mutex **per conto**, nella struct del conto (più parallelismo di un lock globale).
- Il trasferimento tocca **due** conti → servono due lock → rischio **deadlock** se due thread fanno A→B e B→A.
- Soluzione: **ordine fisso**, si prende sempre prima il conto con `id` minore.
- Caso limite: `from == to` ritorna subito (lo stesso mutex preso due volte si blocca da solo).
- Il **controllo del saldo va fatto con i lock presi**: se fosse prima, due thread potrebbero superare il controllo insieme e rendere il saldo negativo (*check-then-act*).
- `rand_r` con un seed per thread al posto di `rand()`.

E5_conti.c

```c
/* E5_conti.c - trasferimenti concorrenti tra conti, senza deadlock e senza perdere soldi
 * Compilare: gcc -Wall -Wextra -pthread E5_conti.c -o E5
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define N_ACCOUNTS  8
#define N_THREADS   4
#define N_TRANSFERS 100000
#define START_MONEY 1000

typedef struct {
    int             id;          /* l'id definisce l'ORDINE con cui si prendono i lock */
    long            balance;
    pthread_mutex_t lock;
} account_t;

account_t acc[N_ACCOUNTS];

/* ritorna 0 se il trasferimento e' avvenuto, -1 altrimenti */
int transfer(account_t *from, account_t *to, long amount) {
    if (from == to) return -1;                      /* stesso mutex due volte = deadlock! */

    account_t *first  = from->id < to->id ? from : to;
    account_t *second = from->id < to->id ? to : from;

    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);

    int ok = -1;
    if (from->balance >= amount) {                  /* il controllo va FATTO con i lock presi */
        from->balance -= amount;
        to->balance   += amount;
        ok = 0;
    }

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
    return ok;
}

typedef struct {
    unsigned seed;               /* seme PRIVATO: rand() non e' thread-safe, si usa rand_r */
    long     done;               /* trasferimenti riusciti */
} worker_t;

void *worker(void *arg) {
    worker_t *w = (worker_t *)arg;
```

```c
    for (int i = 0; i < N_TRANSFERS; i++) {
        int  a      = rand_r(&w->seed) % N_ACCOUNTS;
        int  b      = rand_r(&w->seed) % N_ACCOUNTS;
        long amount = rand_r(&w->seed) % 200 + 1;
        if (transfer(&acc[a], &acc[b], amount) == 0)
            w->done++;
    }
    return NULL;
}

int main(void) {
    for (int i = 0; i < N_ACCOUNTS; i++) {
        acc[i].id      = i;
        acc[i].balance = START_MONEY;
        pthread_mutex_init(&acc[i].lock, NULL);
    }

    pthread_t tid[N_THREADS];
    worker_t  w[N_THREADS];
    for (int i = 0; i < N_THREADS; i++) {
        w[i].seed = 1000 + i;
        w[i].done = 0;
        pthread_create(&tid[i], NULL, worker, &w[i]);
    }

    long done = 0;
    for (int i = 0; i < N_THREADS; i++) {
        pthread_join(tid[i], NULL);
        done += w[i].done;
    }

    long total = 0;
    for (int i = 0; i < N_ACCOUNTS; i++) total += acc[i].balance;

    printf("trasferimenti riusciti: %ld\n", done);
    printf("totale iniziale = %d, finale = %ld -> %s\n",
           N_ACCOUNTS * START_MONEY, total, total == (long)N_ACCOUNTS * START_MONEY ? "OK" : "ERRORE");
    return 0;
}
```

Output verificato (il numero di trasferimenti riusciti cambia a ogni esecuzione, il totale no)

```
trasferimenti riusciti: 321073
totale iniziale = 8000, finale = 8000 -> OK
```

CONSIGLIO

Se la traccia cambia… *Un thread "revisore" che stampa il totale mentre gli altri trasferiscono*: deve prendere i lock di **tutti** i conti nello stesso ordine (id crescente). *Trylock*: se il secondo lock fallisce, rilascia il primo e riprova.

5.6 E6 — Server TCP: un thread per client e statistiche condivise

NOTA

Traccia. Un server TCP accetta più client contemporaneamente. Ad ogni messaggio ricevuto risponde con il numero di client attualmente connessi e il numero totale di messaggi ricevuti da tutti i client.

Come ragionare

- `accept` in un ciclo infinito; per ogni connessione un thread **detached** (nessuno farà `join`).
- Il descrittore del client si passa **copiato in memoria allocata** (`malloc`), mai `&fd` del ciclo: al giro successivo cambia. Il thread fa `free` della sua copia e `close` della sua socket.
- Statistiche globali (`active`, `messages`) → **mutex** nella stessa struct.
- La risposta si costruisce **dentro** il lock (valori coerenti) e si invia **fuori** (mai I/O di rete con un lock preso).

- `MSG_NOSIGNAL` in `send` evita `SIGPIPE` se il client è già sparito; il ciclo `recv` finisce quando il client chiude (0) o in caso di errore (-1).

E6_server_stats.c

```c
/* E6_server_stats.c - server TCP: un thread per client + statistiche globali protette da mutex
 * Ad ogni messaggio ricevuto risponde con "clienti=<attivi> messaggi=<totale>".
 * Uso:       ./E6 5000     poi:   nc localhost 5000
 * Compilare: gcc -Wall -Wextra -pthread E6_server_stats.c -o E6
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>

/* dati condivisi + mutex che li protegge, insieme */
static struct {
    int             active;      /* client connessi in questo momento */
    long            messages;    /* messaggi ricevuti da tutti i client */
    pthread_mutex_t lock;
} stats = { 0, 0, PTHREAD_MUTEX_INITIALIZER };

static void *handle_client(void *arg) {
    int fd = *(int *)arg;
    free(arg);                                      /* il thread libera la SUA copia */

    pthread_mutex_lock(&stats.lock);
    stats.active++;
    pthread_mutex_unlock(&stats.lock);

    char buf[1024], reply[64];
    ssize_t n;
    while ((n = recv(fd, buf, sizeof(buf), 0)) > 0) {
        pthread_mutex_lock(&stats.lock);
        stats.messages++;
        /* la risposta si costruisce DENTRO il lock (valori coerenti)... */
        snprintf(reply, sizeof(reply), "clienti=%d messaggi=%ld\n", stats.active, stats.messages);
        pthread_mutex_unlock(&stats.lock);

        /* ...ma si invia FUORI: mai I/O di rete con un lock preso */
        if (send(fd, reply, strlen(reply), MSG_NOSIGNAL) < 0) break;
    }

    pthread_mutex_lock(&stats.lock);
    stats.active--;
    pthread_mutex_unlock(&stats.lock);
    close(fd);
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "uso: %s porta\n", argv[0]); return EXIT_FAILURE; }

    int srv = socket(AF_INET, SOCK_STREAM, 0);
    if (srv < 0) { perror("socket"); return EXIT_FAILURE; }
    int yes = 1;
    setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port        = htons(atoi(argv[1]));

    if (bind(srv, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind");   return EXIT_FAILURE; }
    if (listen(srv, 10) < 0)                                   { perror("listen"); return EXIT_FAILURE; }
    printf("in ascolto sulla porta %s\n", argv[1]);
    fflush(stdout);

    for (;;) {
        int fd = accept(srv, NULL, NULL);
        if (fd < 0) { perror("accept"); continue; }

        int *pfd = malloc(sizeof(int));             /* una copia del descrittore PER OGNI client */
```

```
        if (!pfd) { close(fd); continue; }
        *pfd = fd;

        pthread_t tid;
        if (pthread_create(&tid, NULL, handle_client, pfd) != 0) {
            free(pfd); close(fd); continue;
        }
        pthread_detach(tid);
    }
}
```

Verificato con 5 client simultanei

```
$ ./E6 5000        (5 client connessi insieme, ognuno invia un messaggio)
clienti=5 messaggi=1
clienti=5 messaggi=2
clienti=5 messaggi=3
clienti=5 messaggi=4
clienti=5 messaggi=5
(dopo la chiusura dei 5, un nuovo client riceve:  clienti=1 messaggi=6)
```

CONSIGLIO

Se la traccia cambia… *Chat con broadcast*: lista dei client in una struct protetta da mutex; per inviare a tutti prendi il lock e scorri la lista. *Numero massimo di client*: un semaforo inizializzato al massimo, `sem_wait` prima di `accept` /creazione del thread e `sem_post` quando il client esce. *Thread pool*: coda di descrittori come in E2, con N worker fissi.

6. Errori tipici, checklist e test

6.1 I 15 errori che costano più punti

Errore	Sintomo	Rimedio
`pthread_create(..., &i)` con `i` del ciclo	Thread con lo stesso valore	`args[i]` oppure `(void *)(intptr_t)i`
`return &variabile_locale;`	Crash o valori casuali	Campo della struct o `malloc`
`main` termina senza `join`	Output mancante	`join` di tutti i thread
Dato condiviso senza mutex, o mutex diversi	Risultato diverso a ogni esecuzione	Un mutex per dato, **lo stesso** ovunque
`if` al posto di `while` con `cond_wait`	Errori rari, difficili da ripetere	`while (!predicato)`
`signal` senza aver cambiato lo stato	Thread bloccato per sempre	Prima il predicato, poi `signal`
`lock` senza `unlock` su un percorso	Programma bloccato	Uscita unica con `goto out`
Lock in ordine diverso nei thread	Deadlock	Ordine fisso (per id/indirizzo)
Mutex preso prima del semaforo contatore	Deadlock nel produttore/consumatore	Prima `sem_wait` contatore, poi mutex
`sleep` per sincronizzare	Funziona "quasi sempre"	`join` , semafori, condition variable
`rand()` nei thread	Risultati non riproducibili / race	`rand_r(&seed_privato)`
Controllo di una risorsa fuori dal lock	Saldo negativo, buffer oltre il limite	Controllo e modifica sotto lo stesso lock
`&fd` condiviso nel ciclo `accept`	Client scambiati o socket chiusa	`malloc` di una copia per ogni client
`printf` / `recv` / `send` dentro la sezione critica	Programma lento, poco parallelo	Fuori dal lock
Dimenticare `-pthread`	`undefined reference to pthread_create`	`gcc -pthread`

6.2 Checklist prima di consegnare

1. Compila **senza warning** con `-Wall -Wextra -pthread` ?
2. Ogni thread joinable ha il suo `join` (o è detached di proposito)?
3. Gli argomenti dei thread sono distinti e ancora validi quando i thread li leggono?
4. Ogni dato condiviso ha **un** mutex, usato in **tutti** gli accessi (letture comprese)?
5. Ogni `lock` ha il suo `unlock` su **tutti** i percorsi?
6. Ogni `cond_wait` è in un `while` , con il mutex preso, e chi segnala ha prima cambiato lo stato?
7. Più lock → ordine fisso? Più semafori → contatore prima del mutex?
8. Nessun `sleep` /I/O/calcolo lungo dentro le sezioni critiche?
9. Il programma termina sempre: nessun thread resta in attesa per sempre?
10. `malloc` controllati e `free` fatti; `destroy` di mutex, cond, semafori?
11. Controllato il ritorno di `pthread_create` , `malloc` , `sem_init` ?
12. Argomenti da riga di comando verificati (`argc` , valori validi)?

6.3 Come provare che funziona

Cosa fai	Comando
Ripeti 100 volte e confronta le uscite	`for i in $(seq 100); do ./prog; done | sort | uniq -c`
Cerca blocchi (deadlock)	`for i in $(seq 100); do timeout 5 ./prog >/dev/null || echo BLOCCATO; done`
Rileva data race	`gcc -g -O1 -fsanitize=thread -pthread prog.c -o p && ./p`
Ordine dei lock e uso scorretto delle API	`valgrind --tool=helgrind ./prog`
Dove si è bloccato	`gdb -p PID` poi `thread apply all bt`
Perdite di memoria	`valgrind --leak-check=full ./prog`

CONSIGLIO

Per far emergere un bug di concorrenza riduci il problema (2 thread, poche iterazioni) o inserisci un `usleep` tra due passi **solo come test**. Se il programma è corretto, nessun `usleep` deve cambiarne il risultato.

6.4 Da saper spiegare se ti chiedono "perché?"

Domanda	Risposta breve
Perché `while` e non `if` con `cond_wait` ?	Risvegli spuri, e un altro thread può aver reso falsa la condizione tra il risveglio e la ripresa del mutex.
Perché l'ordine dei lock?	Rompe l'attesa circolare, una delle 4 condizioni necessarie al deadlock.
Perché un mutex per conto e non uno globale?	Più parallelismo: trasferimenti su conti diversi non si bloccano a vicenda.
Perché si accumula in locale e si somma a fine lavoro?	Meno contesa sul lock: il mutex si prende una volta sola per thread.
Differenza tra mutex e semaforo?	Il mutex ha un proprietario ed è per la mutua esclusione; il semaforo è un contatore e `post` può farlo qualunque thread.
Perché `join` rende visibili i risultati?	Il `join` sincronizza: tutto ciò che il thread ha scritto prima di terminare è visibile al thread che lo attende.
Joinable o detached?	Joinable se serve il risultato o attendere la fine; detached per lavori indipendenti (un thread per client).

6.5 Appendice — File e Makefile

File	Traccia	Sezione
`00_template.c`	Scheletro di partenza	2.1
`E1_max_parallelo.c`	Massimo e pari con N thread	5.1
`E2_coda_limitata.c`	Produttori/consumatori con coda e chiusura	5.2
`E3_barriera_cond.c`	Barriera con mutex + condition variable	5.3
`E4_anello_semafori.c`	Staffetta A B C con semafori	5.4
`E5_conti.c`	Trasferimenti senza deadlock	5.5
`E6_server_stats.c`	Server TCP, un thread per client	5.6